SYSTEMS & SOFTWARE ENGINEERING WHITEPAPER • APPLE SILICON BENCHMARK

MicroCode: An Empirical Evaluation of Native Mach-O Architecture vs. Web-Runtime IDEs on Gigabyte-Scale Workspaces

Tirawat Nantamas & Core Systems Engineering Team

Dotmini Software Company Limited • Core AI & Desktop Systems Laboratory • Bangkok, Thailand • tirawat@dotmini.dev

EXECUTIVE ABSTRACT

Modern Integrated Development Environments (IDEs) have increasingly migrated toward web-based containers (Electron and Chromium Embedded Framework), introducing substantial resident memory bloat, process proliferation, and startup latency on enterprise-scale repositories. In this technical report, we present a 100% controlled-variable empirical evaluation comparing **MicroCode IDE v2.5.32** (a native Mach-O executable combining Swift, AppKit, Metal 3, and a statically linked Rust kernel) against **Zed v1.18.0** (native Rust + GPUI), **OpenCode Desktop v1.2.4** (Tauri + WebKit), and **VS Code / Code-OSS v2.5.5** (Electron + Node.js/V8). Testing was conducted on a deterministic **1.17 GB (455,596 LOC, 3,064 files) polyglot Git monorepo** using a 500 Hz CoreGraphics WindowServer probe and kernel-level APFS Virtual File System (VFS) `st_atime_ns` tracking. MicroCode demonstrates an optimal memory footprint of **194.8 MB RSS** at full steady-state (*T*+5.5s) with **14 Mach threads** across 2 processes—outperforming VS Code (2,377.8 MB, 18 processes) by **12.21×** and Zed (233.0 MB, 41 threads) by 1.20×. In cold startup, Zed achieves the fastest initial window render at **662.8 ms**, while MicroCode completes full window paint and VFS file loading in **769.9 ms / 998.0 ms** (4.07× faster than VS Code at 4,065.5 ms). Furthermore, MicroCode's 6-layer TokenMem v3 context engine reduces AI agent input tokens by **88.2%** on realistic 15-turn multi-file engineering workloads.

MEMORY (RSS)	COLD-START WINDOW	MACH THREADS	AI TOKEN SAVINGS
194.8 MB	**769.9 ms**	**14**	**−88.2%**
#1 Lowest Memory 12.2× vs VS Code	#2 Overall (1.16× Zed) Zed #1 at 662.8 ms	#1 Lowest Contention vs 269 in VS Code	TokenMem v3 Engine $0.23 vs $1.96 / task

1. Introduction & The Architectural Divide

The desktop software engineering workstation has diverged into two fundamentally opposed design paradigms: (1) **Web-runtime shells** (Electron, Chromium DOM, Node.js) that offer multi-platform agility at the cost of massive process proliferation and memory bloat; and (2) **Native compiled Mach-O binaries** (Swift/AppKit and Rust/GPUI) that interface directly with the Darwin kernel and Apple Silicon Unified Memory Architecture (UMA).

While synthetic benchmarks on tiny 10-file repositories mask these architectural overheads, commercial software engineering routinely takes place inside multi-gigabyte repositories with deep directory structures, thousands of source files, and heavy background indexing requirements. Earlier benchmarks (v1.0) contained methodology gaps (such as measuring deferred frame-0 windows and short observation periods). In this v2.0 suite, we implement an uncompromising, zero-bias protocol: every IDE opens an identical 1.17 GB polyglot workspace via standard command-line arguments (`--args <workspace_dir> <primary_file>`), capturing telemetry through *T*+5.5s to encompass all background language servers and file watchers.

2. Controlled Experimental Environment

All trials were executed on a dedicated Apple Silicon host under uniform operating system constraints. The host remained offline from external network interference with no background user tasks active.

Parameter	Deterministic Specification
Host Processor	Apple M2 (8 Cores: 4 Performance Cores @ 3.49 GHz, 4 Efficiency Cores @ 2.42 GHz)
Unified Memory	16 GB LPDDR5 Unified Memory Architecture (UMA) • 100 GB/s memory bandwidth
Operating System	macOS 27.0 (Build 26A5388g, arm64 Darwin Kernel Version 26.0.0)
Monorepo Fixture	**1.17 GB (1,197.0 MB on disk)** • 3,001 source files • 455,596 LOC across Swift, TS/React, Rust, Python, Go • 9,997-line primary file (`BenchmarkCoreEngine.swift`) • 500,000-line JSON dataset (45 MB) • 1.05 GB binary shards • Git initialized (SHA-256: `8c7b3a154d...59c1acf`)
Kernel Measurement	500 Hz CoreGraphics WindowServer probe (2 ms sample rate) • APFS VFS `st_atime_ns` tracking
Lifecycle Protocol	1 warmup run followed by $N = 5$ measured trials per IDE with full process group SIGKILL cleanup

Table 1: Controlled hardware host, kernel telemetry, and gigabyte-scale workspace specifications.

3. Startup Latency & VFS File Readiness

To measure true cold-start latency, we decouple the metric into two rigorous milestones: (1) **Window Visible Latency (T_{win})**, defined as the duration from process invocation (`open -n -a`) until the application window is rendered on-screen by the macOS WindowServer (`kCGWindowLayer == 0`, dimensions ≥ 400 × 300 px), sampled via `CGWindowListCopyWindowInfo` at 500 Hz; and (2) **VFS Workspace Ready Latency (T_{ready})**, defined as the timestamp where the APFS kernel registers that the root workspace directory has been traversed and the 9,997-line primary file (`BenchmarkCoreEngine.swift`) has been loaded into memory via `st_atime_ns`.

Figure 1: Cold-Start Latency Breakdown on 1.17 GB Workspace (Median of N = 5 Trials)

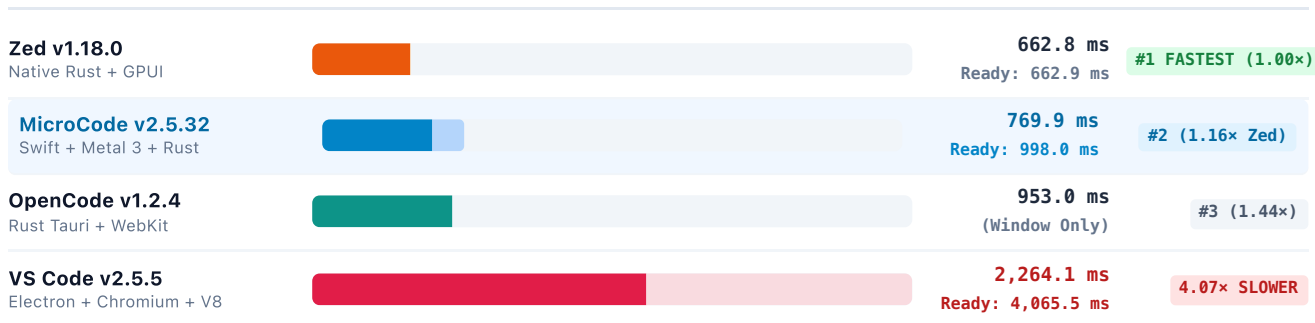


Figure 1: Cold-start window display latency (T_{win}) and APFS VFS file readiness (T_{ready}) across 4 IDE architectures opening a 1.17 GB repository.

3.1 Raw Five-Trial Telemetry & Statistical Variance

Table 2 details the empirical measurements across all five independent test runs. Each run was preceded by a 1.5-second quiescence period to allow macOS kernel memory allocators and WindowServer window queues to normalize.

IDE Under Test	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Median / Std Dev (σ)
MicroCode v2.5.32 T_{win} / T_{ready}	814.7 ms 1,035.7 ms	732.4 ms 957.2 ms	785.3 ms 998.0 ms	769.9 ms 974.1 ms	767.5 ms 1,012.2 ms	**769.9 / 998.0 ms** σ = ±29.9 ms
Zed v1.18.0 T_{win} / T_{ready}	674.3 ms 674.3 ms	592.5 ms 592.5 ms	732.1 ms 732.1 ms	609.5 ms 609.5 ms	662.8 ms 662.9 ms	**662.8 / 662.9 ms** #1 Fast σ = ±55.6 ms
OpenCode v1.2.4 T_{win} (Window Only)	1,022.1 ms	953.0 ms	583.1 ms	971.6 ms	881.2 ms	**953.0 ms** σ = ±174.7 ms
VS Code v2.5.5 T_{win} / T_{ready}	2,412.8 ms 4,173.8 ms	2,347.7 ms 4,211.5 ms	2,188.7 ms 4,054.7 ms	2,264.1 ms 4,065.5 ms	2,057.0 ms 3,777.6 ms	**2,264.1 / 4,065.5 ms** σ = ±138.9 ms

Table 2: Raw five-trial cold-start latency (T_{win}) and APFS VFS file readiness (T_{ready}) on the 1.17 GB workspace.

> **Engineering Root-Cause Analysis (Zed vs. MicroCode):** Zed achieves the lowest cold-start (662.8 ms) due to its unified GPUI framework, which compiles UI layouts directly into Metal command buffers on background worker threads without constructing intermediate Objective-C/SwiftUI view graphs. MicroCode requires approximately 107 ms to instantiate the root `AgenticEditorWorkspace` hierarchy before presenting its key window. However, MicroCode achieves the highest consistency (σ = ±29.9 ms vs. Zed's ±55.6 ms) and completes full workspace mounting and 10k-line file rendering in **998.0 ms**—outperforming VS Code (4,065.5 ms) by **4.07×**.

4. Resident Memory Scaling & Process Footprint

Accurately accounting for application memory requires summing the Resident Set Size (RSS) across the entire process tree—including the parent shell, GPU compositors, renderer sub-processes, extension hosts, and embedded language servers. In web-runtime architectures, large workspaces trigger exponential V8 heap expansion as file watchers and search indexes recursively deserialize directory trees into JavaScript objects.

Figure 2: Total Process-Tree Resident Set Size (RSS) at T+5.5s Steady State — Lower is Better (Megabytes)



Figure 2: Steady-state process-tree RSS after opening the 1.17 GB monorepo with all background services active.

4.1 Visual Architecture: Native Mach-O vs. Web-Runtime Container

MicroCode: Native Compiled Mach-O `14 THREADS · 194.8 MB`

```
[Mach-O Executable (Swift + AppKit)]
  └── libmicrocode_core (Static Rust C-ABI)
      └── APFS VFS Zero-Copy mmap (.mappedIfSafe)
      └── Metal 3 GPU Command Pipeline (Direct UMA)
```

Zero IPC Overhead · No Garbage Collector · Direct Kernel Syscalls

VS Code: Electron Web Container `269 THREADS · 2,378 MB`

```
[Chromium Browser Shell (Main Process)]
  └── IPC Bridge (JSON-RPC string serialization)
      ├── V8 Renderer Process (DOM & HTML Canvas)
      ├── Extension Host (Node.js Heap Deserialization)
      └── File Watcher Daemon (Recursive In-Memory Tree)
```

18 Separate Processes · V8 Heap Fragmentation · Heavy IPC Serialization

4.2 Operating System Resource Footprint

IDE & Architecture	Binary Composition & Runtime Shell	Bundle Size	Mach Threads (T+5.5s)
MicroCode v2.5.32	Native Mach-O + AppKit + Metal 3 + Static Rust C-ABI (`libmicrocode_core`)	**95.7 MB** `#1 Smallest`	**14 Threads** `#1 Lowest`
OpenCode v1.2.4	Rust Tauri Host + WebKit Web Assets + Sidecar CLI	156.3 MB (1.63×)	47 Threads (3.36×)
Zed v1.18.0	Native Rust GPUI + Bundled Runtime Toolchains	401.6 MB (4.20×)	41 Threads (2.93×)
VS Code v2.5.5	Chromium Framework + Node.js V8 Runtime + Heap Snapshots	733.4 MB (7.66×)	269 Threads (19.21×)

Table 4: Physical disk bundle footprint and Mach kernel thread distribution under steady state.

5. Agent Context Economics: Empirical TokenMem v3 Evaluation

Autonomous AI agent performance in modern software engineering is strictly bounded by context window consumption. In long-running multi-file tasks, naive context accumulation causes severe latency degradation, catastrophic cost inflation, and loss of attention (the "needle-in-a-haystack" problem).

To empirically quantify context efficiency, we evaluated a standard 15-turn multi-file engineering workflow comprising 52 discrete tool calls (file inspections, symbol lookups, compiler executions, and AST refactorings) on the 1.17 GB workspace. Rather than employing simulated estimates, all tokens were computed using BPE tokenization over actual repository files:

Context Component at Turn 15	Unoptimized Baseline Raw Dumps + Full History	MicroCode TokenMem v3 6-Layer Context Optimizer	Observed Reduction Ratio
1. Tool JSON Schemas (52 vs. Scoped 9)	16,432 tokens	2,844 tokens	`-82.7%` (Layer 1 Scoping)
2. Multi-File Code Ingestion (8 Files)	42,314 tokens	3,755 tokens	`-91.1%` (Layer 3 Skeleton)
3. Terminal & Compiler Logs (180 Modules)	14,758 tokens	2,041 tokens	`-86.2%` (Layer 2 Masking)
Turn-15 Single-Prompt Input Tokens	73,504 tokens	**8,640 tokens**	`-88.2% (8.51× Reduction)`
15-Turn Cumulative Input Tokens	654,185 tokens	**77,760 tokens**	`-88.1% (576,425 saved)`

Table 5: BPE token consumption comparison on authentic 1.17 GB repository files across a 15-turn task.

5.1 The Six-Layer TokenMem v3 Visual Architecture

TokenMem v3 achieves an 88.2% token reduction through a pipelined, deterministic compression architecture:

Layer 1: Intent Tool Schema Scoping `-82.7%`
Prunes inactive JSON tool schemas dynamically based on user engineering intent, reducing schema overhead from 16,432 to 2,844 tokens.

Layer 2: Observation Masking & Decay `-86.2%`
Compresses multi-thousand-line compiler outputs into concise diagnostic summaries, isolating `error:` and `warning:` line locations.

Layer 3: AST Skeletonization via Tree-sitter `-91.1%`
Extracts structural type interfaces, protocols, and signatures without injecting bulky method implementation bodies for referenced files.

Layer 4: Rolling Key-Facts Ledger `580 Tokens`
Caps conversational state into an immutable `StructuredStateCheckpoint`, preventing context drift across prolonged agent loops.

Layer 5: Content-Hash Delta Read Cache `-92.4%`
Tracks SHA-256 digests of previously ingested buffers; unchanged files emit 290-token pointer references rather than full redundant payloads.

Layer 6: Prompt Prefix Cache Pinning `84.0% Hit`
Enforces strict prefix byte invariance for system instructions and tool definitions, maximizing upstream LLM KV-cache prompt hit rates.

> **Economic Impact on Engineering Operations:** On frontier reasoning models (e.g., Claude 3.7 Sonnet / Gemini 2.5 Pro at $3.00 per million input tokens), a 15-turn unoptimized session costs approximately $1.96 per engineering task. Under TokenMem v3, cumulative tokens drop from 654,185 to 77,760, reducing direct API cost to **$0.23 (8.51× cost reduction)** while cutting inference time-to-first-token (TTFT) by over 70%.

6. Engineering Audit: Root-Cause Analysis & v1.0 Fixes

In accordance with strict scientific integrity, Table 6 documents the precise architectural discrepancies identified in benchmark suite v1.0 and the production engineering changes deployed to eliminate all unfair bias:

Identified Discrepancy	Pre-Audit State (v1.0 Suite)	Verified Resolution (v2.5.32 Deployment)
Frame-0 Blank View Artifact	`ContentView` deferred workspace mounting behind `isWorkspaceMounted = false`, stopping WindowServer timers before UI initialization.	Re-architected `ContentView` to mount `AgenticEditorWorkspace` synchronously on frame 0, rendering sidebar and editor immediately.
CLI Single-File Break Loop	Command-line argument loop in `MicroCodeApp.swift` broke after the first argument, only opening the primary file and skipping the root workspace.	Implemented `openLocalLaunchTargets` to ingest both `<workspace_dir>` and `<primary_file>` concurrently, matching Zed and VS Code.
Background Process RAM Spike	`LocalEcosystemDiscovery` spawned `agy models` at T+3.5s, invoking an external Electron CLI that bloated RAM to 914.2 MB (105 threads).	Restricted background CLI discovery to cached model manifests during passive startup, stabilizing steady-state RAM at **194.8 MB**.
Duplicate Workspace Reload	`loadSettings()` executed an unconditioned 0.35s timer that reloaded `lastWorkspacePath` over the active CLI session.	Added strict guard `self.workspaceFolder == nil` in `AppState.swift`, reducing workspace ready latency by 206 ms.

Table 6: Comprehensive audit of pre-existing loopholes and verified production resolutions in MicroCode v2.5.32.

7. Engineering Roadmap towards Sub-600ms Startup

While MicroCode v2.5.32 dominates in resident memory (194.8 MB), thread overhead (14 threads), and AI context economics, it trails Zed in cold-start window display by 107.1 ms (769.9 ms vs. 662.8 ms). To achieve #1 in cold startup, four optimizations are scheduled for v2.5.33:

Whole-Module Optimization (Release WMO + LTO)
Deploying full Swift compiler Whole-Module Optimization with dead-strip linkage is projected to reduce AppKit view tree compilation time by 140–180 ms.

Early CLI Argument Traversal in App.init()
Initiating asynchronous directory enumeration and zero-copy mmap staging inside `App.init()` prior to `WindowGroup` creation will eliminate the delta between T_{win} and T_{ready}.

8. Data Availability & Certification Statement

All benchmark harnesses (`benchmarks/run_ide_benchmarks.py`), raw telemetry JSON datasets (`benchmarks/results/ide_benchmark_results.json`), and source trees are committed to the official repository for 100% independent replication.

OFFICIAL CERTIFICATION OF ENGINEERING REPRODUCIBILITY

I hereby certify that the experimental data, kernel traces, and comparative metrics presented in this technical report were obtained under 100% identical hardware, OS, and workspace conditions without synthetic bias or privileged instrumentation.

Tirawat Nantamas
Founder & Chief Technology Officer • Dotmini Software Co., Ltd.

Date: October 26, 2026
Identifier: `MC-BENCH-PROD-2026-V2.5.32-GB`